BSON (Binary JSON) - Zero to Hero

Before BSON, let's quickly understand the journey.

XML SOLVED DATA SHARING

XML helped different systems exchange data.

But XML had problems:

- too many tags
- larger payloads
- more bandwidth usage
- complex parsing

So JSON arrived.

JSON SOLVED MANY XML PROBLEMS

JSON was:

- simpler
- smaller
- easier to read
- easier to parse

Example:

```
{
    "name": "Chandan",
    "age": 27
}
```

Developers loved it.

Web APIs loved it.

The internet loved it.

But databases had a different problem.

And that's where BSON enters the story.

Why BSON Was Created

JSON is great for humans.

But computers don't naturally think in text.

Computers work in binary.

When a JSON document arrives:

```
{
    "name": "Chandan",
    "age": 27
}
```

the database must:

1. Read text
2. Parse text
3. Figure out data types
4. Convert into internal objects

For a few documents, no problem.

For millions of documents, every extra step costs CPU and memory.

MongoDB wanted something better.

The goal was:

- keep JSON-like structure
- support more data types
- make processing easier for databases

So BSON was created.

What Is BSON?

BSON stands for:

Binary JSON

Officially, BSON is a binary-encoded serialization format used by MongoDB.

Think of it like this:

JSON is for humans.

BSON is for computers and databases.

BSON Uses The Same Structure As JSON

JSON:

```
{
    "name": "Chandan",
    "age": 27
}
```

BSON:

Still stores:

```
name -> Chandan
age  -> 27
```

Same key-value idea.

But instead of storing it as text, BSON stores it as binary bytes.

How BSON Stores Data

High-level BSON structure:

```
[Document Size]

[Type][Field Name][Value]

[Type][Field Name][Value]

[Type][Field Name][Value]

[Document End]
```

Notice something important.

BSON stores:

- document size
- field type
- field name
- field value

This extra information helps MongoDB process data faster.

Real Example

JSON:

```
{
    "name":"Chandan",
    "age":27
}
```

BSON internally looks something like:

```
20 00 00 00       -> Document Size

02                -> String Type
6e 61 6d 65 00    -> "name"
08 00 00 00       -> String Length
43 68 61 6e 64 61 6e 00  -> "Chandan"

10                -> Int32 Type
61 67 65 00       -> "age"
1b 00 00 00       -> 27

00                -> Document End
```

Don't memorize bytes.

Understand the pattern.

The Important Observation

Look at this field:

```
{
    "age":27
}
```

JSON stores:

```
age -> 27
```

BSON stores:

```
Type = Int32

Field = age

Value = 27
```

MongoDB instantly knows:

- field name
- field type
- field value

No guessing needed.

BSON Supports More Data Types

This is one of BSON's biggest advantages.

JSON supports only:

- String
- Number
- Boolean
- Array
- Object
- Null

That's it.

But databases need much more.

BSON supports:

- Date
- Timestamp
- Binary Data
- ObjectId
- UUID
- Decimal128
- Int32
- Int64
- Regular Expressions

and more.

Example: Date Problem In JSON

JSON:

```
{
    "createdAt":"2026-06-11"
}
```

Question:

Is this:

```
String?
Date?
Timestamp?
```

JSON doesn't know.

For JSON it's just a string.

The application must convert it.

BSON Solves This

BSON stores:

```
Type = Date

Field = createdAt

Value = 2026-06-11
```

MongoDB immediately understands it is a Date.

No conversion needed.

ObjectId - MongoDB's Favorite Data Type

Every MongoDB document usually has:

```
{
   "_id": ObjectId(...)
}
```

ObjectId is a BSON data type.

JSON doesn't have ObjectId.

MongoDB needs it for unique document identification.

This is one reason BSON fits MongoDB perfectly.

Why Document Size Is Stored

Notice the first bytes:

```
20 00 00 00
```

This represents document size.

Suppose MongoDB sees:

```
Document Size = 32 bytes
```

Now it already knows where the document ends.

It doesn't need to scan character by character.

This helps document traversal and storage engine operations.

Is BSON Smaller Than JSON?

Many people think:

> BSON is binary, so it must be smaller.

Not always.

Sometimes BSON is actually larger.

Why?

Because BSON stores:

- type information
- length information
- metadata

Extra metadata means extra bytes.

So BSON was NOT created mainly to reduce size.

Then Why Use BSON?

Because BSON gives:

- native database types
- efficient document storage
- easier processing
- easier querying

- easier indexing
- easier sorting

For databases, these advantages are more important than saving a few bytes.

BSON vs JSON

Feature	JSON	BSON
Human Readable	Yes	No
Text Based	Yes	No
Binary Format	No	Yes
Native Date Support	No	Yes
ObjectId Support	No	Yes
API Friendly	Excellent	Rarely Used
Database Friendly	Good	Excellent
Easy To Debug	Yes	No

BSON vs XML

Feature	XML	BSON
Human Readable	Yes	No
Verbose	Very High	Low
Binary	No	Yes
Parsing Cost	Higher	Lower
Database Storage	Poor	Excellent
Modern Database Usage	Rare	Common in MongoDB

Where BSON Is Used

Mostly in:

- MongoDB document storage
- MongoDB drivers
- database communication
- replication
- internal database operations

Even when you write JSON-like documents in MongoDB:

```
{
    "name":"Chandan"
}
```

MongoDB stores them internally as BSON.

Interview Questions

WHY WAS BSON CREATED?

JSON is human-friendly but databases need richer data types and efficient processing. BSON was created to provide a binary representation of JSON with support for additional database-friendly types.

WHY DOES MONGODB USE BSON?

Because BSON supports:

- ObjectId
- Date
- Binary Data
- Timestamp
- efficient document processing

which are essential for databases.

IS BSON ALWAYS SMALLER THAN JSON?

No.

BSON can sometimes be larger because it stores metadata, type information, and length information.

WHY CAN BSON BE FASTER THAN JSON?

Because BSON stores:

- field type information
- document length
- native database types

This reduces parsing and type inference work for MongoDB.

Final Summary

XML optimized **data sharing**.

JSON optimized **web communication**.

BSON optimized **database storage and processing**.

That's why:

JSON is loved by APIs.

BSON is loved by MongoDB. 🔥